Module 4: LLM Pre-training – Unified Text-to-Text Discriminators

Introduction to Advanced LLM Pre-training

This module delves into advanced paradigms for pre-training Large Language Models (LLMs), moving beyond foundational concepts to explore cutting-edge techniques that drive state-of-the-art performance in Conversational AI.

We will examine two influential approaches:

- **T5 (Text-to-Text Transfer Transformer):** Unifies diverse NLP tasks into a single framework.
- **ELECTRA (Efficiently Learning an Encoder that Classifies Token Replacements Accurately):** Introduces a highly efficient discriminative pre-training method.

These papers are part of the *"Journey through Key LM Papers"* that underpins the field of Conversational AI and LLMs.

Recurrent Neural Networks

Recurrent Neuron

- x_t: Input at time t
- h_{t-1}: State at time $t-1$

$$h_t = f(W_h h_{t-1} + W_x x_t)$$

Unfolding an RNN

$$h_t = f(W_h h_{t-1} + W_x x_t)$$

Weights shared over time!

Part 1: The Text-to-Text Transfer Transformer (T5)

The T5 paper, titled *"Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer,"* provides a comprehensive study of transfer learning techniques for NLP. It is considered **Paper 4 – T5 Prompting For All** in the course's journey through key LM papers.

Core Idea: Unified Text-to-Text Framework

- T5 introduces a **unified framework** that converts all text-based language problems into a text-to-text format.
- Any NLP task is reframed as taking text as input and producing text as output.
- This unification allows for direct application of the same model, objective, training procedure, and decoding process to every task considered.
- It covers a wide variety of English-based NLP problems, including:
  - **Translation:** ‘‘`translate English to German: That is good.`’’ → ‘‘`Das ist gut.`’’
  - **Question Answering**
  - **Summarization:** ‘‘`summarize: [document]`’’ → ‘‘`[summary]`’’
  - **Text Classification:** ‘‘`mnli premise: [premise] hypothesis: [hypothesis]`’’ → ‘‘`entailment`’’
  - **Regression tasks:** e.g., STS-B cast as multi-class classification by converting scores to strings (e.g., ‘‘`3.8`’’).
- The model itself is named the **Text-to-Text Transfer Transformer (T5)**.

Model Architecture

- T5 models are based on the Transformer architecture.
- The primary structure is an **encoder-decoder Transformer**.
  - The encoder uses a **fully-visible attention mask**, attending to the entire input sequence.
  - The decoder uses a **causal (autoregressive) attention mask**, preventing it from seeing future tokens during output generation.
- T5's systematic study compared various architectural variants:
  - **Standard encoder-decoder Transformer**
  - **Language Model (LM):** A single Transformer stack with causal masking throughout, concatenating input and target.
  - **Prefix LM:** A single Transformer stack with fully-visible masking over input (prefix) and causal masking over target.
- **Findings:** Encoder-decoder with a denoising objective performed best.
- Baseline model: Encoder + decoder stacks similar to $\text{BERT}_{\text{BASE}}$, totaling ∼220M parameters.
- **Parameter sharing** across encoder and decoder reduced parameter count with minimal performance loss.

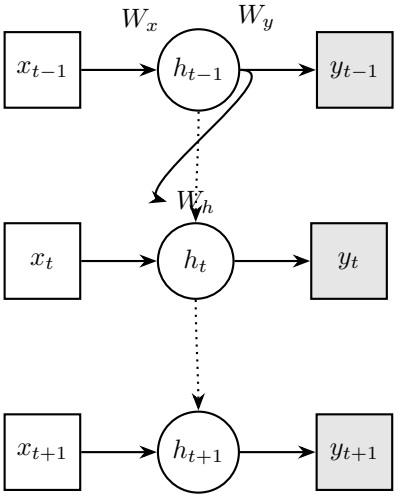
Pre-training Objectives: A Systematic Study

- Central focus: exploring **unsupervised objectives**.
- **Denoising objectives** consistently outperformed language modeling and deshuffling for pre-training.
- Baseline denoising objective:
  1. Randomly sample and drop out 15% of tokens in input.
  2. Replace consecutive spans of dropped-out tokens with a unique **sentinel token**.
  3. Train the model to predict the dropped-out spans, delimited by sentinel tokens.
  4. This is efficient due to shorter target sequences.
- Other denoising variants:
  - **BERT-style:** Reconstruct full sequence, with 15% tokens masked (90% `[MASK]`, 10% random token).
  - **MASS-style:** Replace 15% of tokens with a mask token, reconstruct full sequence.
  - **Dropping corrupted tokens:** Corrupted tokens removed, model reconstructs them in order.
- **Corruption Rate:** 15% effective (as in BERT); 50% significantly degraded performance.
- **Corrupting Spans:** Corrupting contiguous spans (avg. span length 3, 15% corruption) outperformed i.i.d. corruption, with added speed advantages.

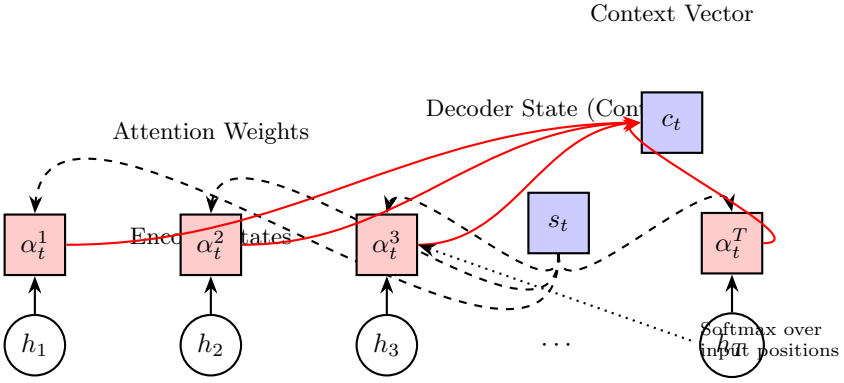
Data Set: The Colossal Clean Crawled Corpus (C4)

- T5 introduced **C4**, a dataset of hundreds of gigabytes of clean English text.
- Derived from Common Crawl, which produces ∼20TB of web text monthly.
- Extensive heuristic filtering was applied to improve quality:
  - Retained lines ending in terminal punctuation.
  - Discarded pages with fewer than 5 sentences or lines with fewer than 3 words.
  - Removed pages with "bad words" or placeholder text (e.g., `lorem ipsum`).
  - Filtered lines containing code-like artifacts (e.g., "Javascript", curly brackets { }).

  - Deduplicated: discarded repeated three-sentence spans.
  - Filtered non-English content using `langdetect` with $p > 0.99$.
- Base dataset size $\approx$ 750 GB.
- Heuristic filtering was crucial: unfiltered C4 degraded performance across tasks.
- Larger and diverse datasets improve generalization, while small datasets repeated too often risk memorization.



$\vdots$



Disadvantages of Traditional RNNs

- After initial popularity, people started moving away
- RNNs / LSTMs / GRUs have disadvantages:
  - Sequential Dependency
  - Not good for GPUs
- Its time to pay attention!

Attention Mechanism for Deep Learning

- Consider an input (or intermediate) sequence or image
- Consider an upper level representation, which can choose where to look, by assigning a weight or probability to each input position, applied at each position

Softmax over lower locations conditioned on context at lower and higher locations

Lower-level layer

Training Strategies and Scaling

- Principle: **"Bigger is Better"** (scaling model size, training steps, and data).
- **Model Sizes:** Small (60M), Base (220M), Large (770M), 3B, and 11B parameters.
  - Best performance: 11B parameters across all tasks.
- **Longer Training:** $\sim$1M steps with batch size 2^{11} ($\sim$1 trillion tokens) improves performance.
- **Multi-task Pre-training + Fine-tuning:** Combining supervised + unsupervised tasks during pre-training achieved results comparable to unsupervised-only, with added monitoring benefits.
- **Fine-tuning Methods:** Updating all parameters consistently outperformed partial methods (e.g., adapters, gradual unfreezing).
- **Ensembling:** Multiple models combined significantly outperform single models.

Key Takeaways for T5

- Unified text-to-text framework enables a single model for diverse NLP tasks.
- Encoder-decoder Transformer with span corruption is highly effective.
- C4 dataset illustrates importance of large, clean, and diverse pre-training corpora.
- Scaling (size, duration, ensembles) yields substantial gains.
- T5 achieved state-of-the-art results on 18 of 24 benchmarks (GLUE, SuperGLUE, SQuAD, CNN/DailyMail).

Part 2: ELECTRA: Efficient Discriminative Pre-training

Core Idea: Replaced Token Detection

- Novel pre-training task: replaced token detection.
- Instead of masking tokens (BERT-style), tokens are **replaced** with plausible alternatives sampled from a generator.
- The discriminator is trained to predict whether each token is *real* or *fake*.

Contrast with Masked Language Modeling (MLM)

- MLM (BERT): model only learns from 15% of masked tokens.
- ELECTRA: model learns from **all tokens**, making training more efficient.
- Avoids pre-train/fine-tune mismatch caused by artificial [MASK] tokens in BERT.

Model Architecture and Training

- Two networks: Generator (G) and Discriminator (D), both Transformer encoders.
- **Generator (G):** performs MLM, sampling plausible tokens.
- **Discriminator (D):** binary classification (real vs fake tokens).
- Joint loss: $L_{MLM}(x, \theta_G) + \lambda L_{Disc}(x, \theta_D)$.
- Generator trained with maximum likelihood (not adversarially).
- After pre-training, discard G; fine-tune D (the ELECTRA model).

Efficiency and Performance Gains

- Outperforms MLM-based methods (BERT, XLNet) at same compute.
- **Small-scale:** ELECTRA-Small (1 GPU, 4 days) scored +5 GLUE over BERT-Small, even surpassing GPT (30x more compute).
- **Large-scale:** ELECTRA-Large rivals or exceeds RoBERTa/XLNet at less than 1/4 compute.
- Key advantage: learns from **all input tokens**.

Model Extensions and Training Algorithms

- **Weight Sharing:** beneficial between generator and discriminator embeddings.
- **Smaller Generators:** optimal when G is 1/4 to 1/2 size of D.
- **Training Strategies:** joint training most effective; two-stage or adversarial variants underperform.

Key Takeaways for ELECTRA

- Replaced token detection is efficient and powerful for representation learning.
- Discriminative pre-training scales better in compute/parameter efficiency than generative MLM.
- Strong performance across scales makes ELECTRA more accessible for limited compute environments.

T5: Unified NLP Tasks via Text-to-Text Framework

T5 unifies Natural Language Processing (NLP) tasks by adopting a **"text-to-text"** framework, which converts all text-based language problems into a consistent format where the model takes text as input and produces new text as output. This unified approach allows the same model, objective, training procedure, and decoding process to be applied across a wide variety of tasks.

How T5 Unifies NLP Tasks

- **Unified Input/Output Format:**
  - Every NLP problem, whether translation, question answering, summarization, or classification, is reframed as generating a text string given an input text string.
  - Example: Translating "That is good." to German:

        * Input: `translate English to German: That is good.`
        * Output: `Das ist gut.`
    - Text classification tasks, such as CoLA (sentence acceptability), are converted to text generation, outputting a single word like "acceptable" or "not acceptable".
    - Regression tasks, e.g., STS-B similarity scoring, generate string representations of numerical scores (e.g., "2.6").
    - Winograd tasks highlight ambiguous pronouns in input, predicting the referring noun as output.

- **Consistent Model Architecture:**
    - All T5 models use the Transformer architecture, primarily the standard encoder-decoder Transformer, which proved most effective for text-to-text tasks.

- **Common Training Objective:**
    - Maximum likelihood objective during training using *teacher forcing* and cross-entropy loss.
    - Pre-training uses a denoising objective:
        * Randomly select and drop out 15% of tokens in the input.
        * Replace consecutive dropped-out spans with unique sentinel tokens.
        * Model trained to reconstruct the dropped spans as target output.

- **Task-Specific Prefixes:**
    - Prepend input with a task-specific text prefix to guide the model.
    - Examples:
        * Summarization: `summarize: [document]`
        * Natural Language Inference: `mnli premise: [premise] hypothesis: [hypothesis]`

This unified approach simplifies development and application across diverse NLP problems by treating them all as variations of text generation.

Intersection of Foundational AI Concepts and Practical Applications

In the **CSE 449: Conversational AI** course, foundational AI concepts and practical applications intersect through a "learn by building" approach, equipping students with both theoretical understanding and hands-on experience to develop advanced conversational AI systems.

Foundational Concepts

- **Defining AI and Conversational AI:**
  - Artificial Intelligence: "intelligence exhibited by machines or software".
  - Conversational AI: enabling machines to have natural conversations.
  - Positioned as a subarea of NLP, often intertwined with Machine Learning (ML).
- **Understanding Underpinnings and State-of-the-Art Techniques:**
  - **T5 (Text-to-Text Transfer Transformer)**: Unifies NLP tasks with text-to-text framework, uses encoder-decoder Transformer, denoising pre-training, and task-specific prefixes.
  - **ELECTRA**: Sample-efficient pre-training via *replaced token detection*, predicting whether tokens in corrupted input are real or replaced; more compute-efficient than MLM approaches like BERT.

Applying Knowledge through Projects

- **Hands-on "Build" Philosophy:** Immediate translation of theory to practice.
- Tools: Jac/Python, OLlama PyTorch/TensorFlow, HuggingFace.
- Activities:
  - Project/Coding Activities
  - Individual Coding Warm-Ups
  - Guided tutorials in a "code-a-long" style
- Goal: Develop "Good, Useful, and Working" real-world applications, e.g., AI for grocery finding, housing searches, recipe guidance, Spotify playlist curation, mental health chatbots.

Instructor Expertise

- Professor Jason Mars: CS Professor at University of Michigan, 30 patents, 80 publications, serial AI founder.
- Guides students through real-world scenarios to develop career-relevant skills.
- Emphasis on scientific intuition rather than solely foundational theory.

Summary

The course teaches students to invent, design, and build conversational AI systems, fostering deep practical understanding alongside theoretical knowledge.

- **A Comprehensive Guide to Pre-training LLMs**
  This article from Analytics Vidhya offers a step-by-step breakdown of the LLM pre-training process, emphasizing data collection, preprocessing, and model training techniques.
  `https://www.analyticsvidhya.com/blog/2025/02/llm-pre-training/`

- **Understanding LLM Pre-training: Teaching Machines to Think**
  A Medium post that introduces the concept of pre-training, explaining how models learn language patterns by predicting the next token in a sequence.
  `https://medium.com/@tungvu_37498/understanding-llm-pre-training-teaching-machines-to-t`

Technical Deep Dives

- **Comprehensive Guide to LLM Pre-training Steps**
  Weights & Biases provides an in-depth look at the critical pre-training steps for LLMs, including setting hyperparameters, batch sizes, and managing compute resources.
  `https://wandb.ai/site/articles/training-llms/pre-training-steps/`

- **PreTraining LLMs - AI Engineering Academy**
  This resource covers the theory behind fine-tuning and pre-training LLMs, discussing the use of self-supervision and transformer models.
  `https://aiengineering.academy/LLM/TheoryBehindFinetuning/PreTrain/`

Practical Implementation

- **Pretraining Your Own Large Model from Scratch**
  SwanLab Docs offers a hands-on guide to pretraining a large language model from scratch using a Wikipedia dataset, with instructions on leveraging cloud GPUs.
  `https://docs.swanlab.cn/en/examples/pretrain_llm.html`

- **Pretraining: Breaking Down the Modern LLM Training Pipeline**
  Comet's article explains the evolution of LLM pretraining, covering methods like instruction-augmented pretraining and reinforcement learning.
  `https://www.comet.com/site/blog/pretraining/`

Academic Resources

- **Lecture 11: Pre-training and Large Language Models (LLMs)**
  A lecture slide deck from COMP3361 that delves into the concepts of large

language model pretraining.
https://taoyds.github.io/assets/courses/COMP3361-lec11.pdf

- **Pretraining LLMs - DeepLearning.AI**
  DeepLearning.AI offers a short course that provides in-depth knowledge of the steps to pretrain an LLM, encompassing data preparation, model configuration, and performance assessment.
  https://www.deeplearning.ai/short-courses/pretraining-llms/

Visual Learning

- **Stanford CS229: Building Large Language Models**
  A video lecture that covers both pretraining (language modeling) and post-training (SFT/RLHF), exploring common practices in data preparation and model training.
  https://www.youtube.com/watch?v=9vM4p9NN0Ts

Video Visit the URL below to view a video:

https://www.youtube.com/embed/5sLYAQS9sWQ